banrepstats: An R Package for Programmatic Access to Colombian Central Bank Economic Data

1st Fernando Salcedo-Mejía
School of Digital Transformation
Data Science Program
Universidad Tecnológica de Bolívar
Cartagena, Colombia
fsalcedo@utb.edu.co

2nd Antonio Perez-Torres
GIDTEC
Universidad Politécnica Salesiana
Cuenca, Ecuador
jperezt@ups.edu.ec

3rd Cesar Villoria-Núñez
School of Digital Transformation
Universidad Tecnológica de Bolívar
Cartagena, Colombia
cesarvilorian@utb.edu.co

***Abstract*—Banco de la República, Colombia's central bank, publishes more than 250 macroeconomic time series through a web portal, but exposes no documented, stable API for programmatic consumption, forcing analysts into manual downloads or brittle screen-scraping. We present banrepstats, an open-source R package that discovers the bank's indicator catalog, downloads any indicator as a tidy `data.table`, and interoperates with modern time-series forecasting tooling. This paper describes the package's architecture, the practical obstacles encountered while reverse-engineering an undocumented JSON endpoint (silent request filtering, inconsistent timestamp encoding, non-unique indicator identifiers, and an incomplete TLS certificate chain), and how each was resolved. We validate the package by downloading the complete catalog (61 indicators, 256 series, 460,461 observations) and by comparing 42 series against the portal's official CSV exports, which agree on every date and value of the 38,858 observations compared. Finally, we illustrate the integration with forecasting tools by retrieving the national unemployment rate and GDP growth and fitting ARIMA, exponential smoothing, Prophet and neural network models. banrepstats is available under the MIT license on GitHub.**

***Index Terms*—open data, R package, central bank statistics, time series, reproducible research, forecasting, API reverse-engineering, Open Government Data**

I. Introduction

Reproducible economic analysis relies on the programmatic retrieval of official statistics, mitigating the friction and errors inherent to manual, one-off downloads. Central banks are a primary source of such statistics, and several have R packages that wrap their public APIs – for example, `rbcb` for Brazil's Banco Central. Banco de la República (Banrep), Colombia's central bank, publishes 61 indicators, comprising 256 time series, covering prices, interest rates, exchange rates, and the real economy through its SUAMECA statistics portal. Two access paths exist: a legacy SDMX endpoint with a small, hand-curated set of flows, and the JSON API that backs the SUAMECA web application itself, which mirrors the full indicator catalog shown to human users but is not documented as a public API.

This paper describes banrepstats, an R package that targets the latter endpoint to give R users tidy, catalog-driven access to the full breadth of Banrep's published statistics. Our contribution is threefold: (1) a small, composable package design – catalog discovery, keyword search, data retrieval, and plotting – built entirely on tidy data.table semantics; (2) a documented account of the reverse-engineering obstacles inherent to consuming an undocumented internal API, which we believe generalizes to similar central-bank and government open-data efforts in the region; and (3) an empirical validation of the package's coverage and correctness against the official exports of the portal, together with the testing and monitoring strategy that keeps it working as the API evolves.

II. Related Work

The `rsdmx` package [1] provides generic access to any SDMX-compliant statistical API, and an earlier iteration of this project used it against Banrep's SDMX endpoint. However, that endpoint only exposes a small, fixed set of flows, requires manually curating a flow-id table, and returns XML that is comparatively expensive to parse. Country-specific central-bank wrappers such as `rbcb` [2] follow a similar pattern to banrepstats – a thin, idiomatic R layer over a national statistics API – but no equivalent package existed for Banco de la República prior to this work. The same pattern recurs at the international-organization level: `wbstats` [3] wraps the World Bank's open data API, and `eurostat` [4] wraps Eurostat's SDMX service for the European Union. Across all of these, catalog discovery, code-based lookup, and tidy output form a recognizable convention that banrepstats deliberately follows. This effort aligns with the broader Open Government Data (OGD) movement, highlighting the necessity of community-driven tooling when public institutions have yet to provide officially documented APIs. More broadly, banrepstats follows the tidy-data philosophy popularized by the tidyverse and `data.table` [5] ecosystems, and targets the tidyverts forecasting stack (`tsibble`/`fable`) [6] as its natural downstream consumer.

III. Package Architecture

banrepstats exposes four functions and one bundled dataset, summarized in Table I. The design separates *catalog discovery* from *data retrieval*: a caller first resolves a human-readable indicator to a stable code (via

`catalogo_banrep()` or `search_banrep()`), then passes that code to `banrep_data()` to download the series.

TABLE I
CORE FUNCTIONS EXPORTED BY BANREPSTATS

Function	Purpose
`catalogo_banrep()`	Downloads and flattens the live indicator menu tree into one row per indicator.
`search_banrep()`	Case- and accent-insensitive keyword search over topic names, indicator names and codes.
`banrep_data()`	Downloads one or more indicators as a single tidy `data.table`, optionally restricted to a date range.
`series_plot()`	Plots the `data.table` returned by `banrep_data()` with `ggplot2` [9] and a consistent institutional palette.
`banrep_cache`	Offline snapshot of the catalog, avoiding a network call for lookups.

Internally, the SUAMECA catalog is served as a recursively nested JSON menu tree (topic → sub-topic → indicator, up to four levels deep in practice). `catalogo_banrep()` flattens this tree with a depth-agnostic recursive function that walks each node's `menuHijos` (child menu) field and combines the results with `data.table::rbindlist()`, so the package does not assume a fixed number of levels – a real constraint, since a fixed-depth `tidyr::unnest()` chain, our first approach, silently drops indicators on shallower branches and errors on deeper ones. Once an indicator code is resolved, `banrep_data()` requests its series data and reshapes the (possibly multi-series) JSON response into long format, one row per observation.

All requests go through a single layer built on `httr2` [10] that sets a timeout and a `User-Agent`, retries temporary HTTP errors (429, 502, 503, 504) with exponential back-off, and checks every response against the fields the package relies on. Because the API is undocumented, this last check matters most: if Banrep changes the shape of a response, the package stops with an explicit "the API may have changed" error that names the missing fields, rather than failing later with an unrelated message or, worse, returning silently incomplete data.

IV. Implementation Challenges

Consuming an internal, undocumented API surfaced four issues that would not arise against a documented public API.

A. Silent Request Filtering

Requests to the data endpoint without an HTTP `Referer` header matching the SUAMECA web application return `HTTP 200` with an empty body, rather than an error. This is consistent with a reverse-proxy rule that distinguishes browser traffic from direct scripting without breaking the site's own client. We resolved this by attaching a static `Referer` header to every data request; because the failure mode is a silent empty response rather than an error status, this class of bug is easy to miss during development, so the package now reports an empty body as an explicit error and the test suite asserts non-empty results.

B. Timestamp Encoding

Observation dates arrive as Unix epoch milliseconds representing midnight in Bogotá local time (UTC−5), rather than UTC midnight. Converting these directly with `as.Date(ms / 86400000)` leaves a fractional day residue (approximately +0.208 days) baked into the resulting `Date` object: it prints correctly, because `format()` truncates the fractional part, but compares and sorts incorrectly, since the underlying numeric value is not a whole number. The fix is to floor the day count before constructing the `Date`, recovering the intended calendar day unambiguously. Section V confirms that every converted date matches the portal's own exports.

C. Non-unique Indicator Codes

An early version of the catalog used the menu's internal category code (e.g. `"PRECIOS_E_INFLACION"`) as the indicator identifier. This code is shared by every indicator under the same topic, so it does not uniquely identify a series and forces callers to rely on catalog order. We instead use each indicator's `urlAcceso` field – a human-readable slug assigned by Banrep to every leaf indicator (e.g. `"inflacion_y_meta"`) – as the public identifier. In the current catalog all 61 slugs are unique, and all 61 slugs in the catalog snapshot bundled with the package in September 2026 were still present, unchanged, seven weeks later. The field is not entirely stable, however: grouping nodes of the menu, which have no data of their own, used to carry an empty `urlAcceso` and now carry a single blank space, which briefly made 44 grouping nodes look like indicators. The package now treats any blank slug as a grouping node, and the weekly monitoring described in Section V is designed to catch changes of this kind.

D. Incomplete TLS Certificate Chain

The SUAMECA server does not send the intermediate certificate that links its certificate to a trusted root. Browsers usually recover the missing link on their own, but `libcurl`, on which `httr2` relies, refuses the connection on systems whose certificate store lacks that intermediate. Rather than disabling certificate verification, the package ships the missing intermediate certificate and, on an SSL failure, retries the request supplying it, so verification stays enabled throughout; later requests in the same session use it directly.

V. Validation

We assessed the package along three dimensions. The coverage and agreement checks are reproducible with the scripts in the `validation/` folder of the replication materials that accompany this paper, which are distributed separately from the package; the results below were obtained on October 24, 2026.

A. Coverage

We requested every indicator in the live catalog with `banrep_data()`. All 61 indicators were retrieved successfully, yielding 256 series and 460,461 observations at five frequencies: 25 indicators contain daily series, 21 monthly, 9 quarterly, 5 annual and 3 weekly. No series contained duplicated dates, and every date was a whole calendar day.

B. Agreement with Official Exports

For a sample of 12 indicators spanning the five frequencies, we downloaded the CSV files that the SUAMECA portal exports for human users and compared them, series by series and date by date, with the output of `banrep_data()`. A value counts as a match when it agrees with the API value up to the rounding of the export. Table II summarizes the result: the two sources contain exactly the same dates, and all 38,858 values agree, with a maximum absolute difference of zero.

TABLE II
AGREEMENT BETWEEN BANREPSTATS AND OFFICIAL SUAMECA EXPORTS

Frequency	Indic.	Series	Obs.	Match (%)
Daily	3	13	26,534	100
Weekly	1	3	5,709	100
Monthly	5	17	5,254	100
Quarterly	3	8	932	100
Annual	1	1	429	100
Total	12[a]	42	38,858	100

[a]One indicator mixes a monthly and an annual series.

C. Testing and Monitoring

The package has 32 tests (87 expectations) built on `testthat` [11]. Twenty-six of them run offline against small JSON fixtures that mirror the API responses, covering catalog flattening, date conversion, missing values, empty and malformed responses, HTTP errors, the retry policy and the TLS fallback; the remaining six run against the live service. Together they cover 94% of the source code, as measured with `covr` [12]. In the package repository, continuous integration runs `R CMD check` on Linux, Windows and macOS for every push and pull request, and a weekly job runs the live tests against the service, including one that checks that every code in `banrep_cache` still exists in the live catalog.

VI. Availability and Installation

banrepstats is free and open-source software released under the MIT license; this paper describes version 1.1.0. The source code, issue tracker and documentation are hosted at https://github.com/fersalme/banrepstats. Since the package is not yet on CRAN, it is installed from GitHub:

```
remotes::install_github("fersalme/banrepstats")
```

Adding `build_vignettes = TRUE` also builds the vignette, a longer walkthrough of catalog browsing, data retrieval, plotting and the tidyverts integration shown in Section VII.

VII. Illustrative Examples

A. Browsing the Catalog

A caller who does not already know an indicator's code can search the catalog by keyword. The search ignores case and accents, so Listing 1 finds every indicator whose topic, name or code mentions "inflación" (all seven belong to the "Precios e inflación" topic); Table III shows the first rows of the result, including the `idNombreSerie` code that `banrep_data()` expects.

Listing 1. Searching the catalog by keyword

```
search_banrep("inflacion")[, .(NombreSerie,
    idNombreSerie)]
```

TABLE III
FIRST THREE OF THE SEVEN ROWS RETURNED BY LISTING 1

NombreSerie	idNombreSerie
Inflación y meta	inflacion_y_meta
IPC_Índice de Precios al Consumidor	indice_de_precios_al_consumidor_ipc
IPP_Índice de Precios del Productor	indice_de_precios_al_productor_ipp

B. Downloading and Plotting a Series

Once the code is known, `banrep_data()` downloads the indicator and `series_plot()` renders it directly, as shown in Listing 2 and Fig. 1. A single indicator code can still resolve to more than one related series – here, `"inflacion_y_meta"` returns both the inflation target and realized annual inflation – which `series_plot()` distinguishes by color. The optional `start` and `end` arguments of `banrep_data()` restrict the result to a date range.

Listing 2. Downloading and plotting an indicator

```
precios <- banrep_data(indicator = "inflacion_y_meta
    ")
series_plot(precios)
```

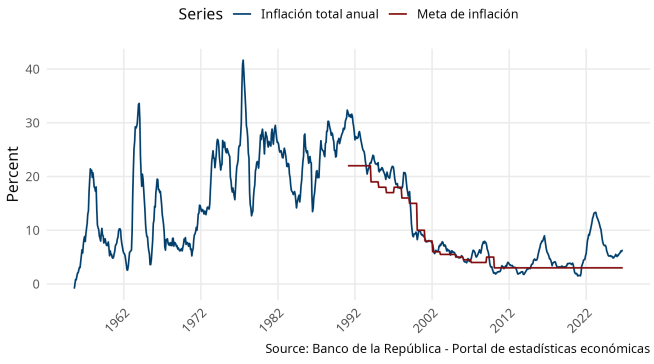


Fig. 1. Output of Listing 2: Colombia's inflation target and realized annual inflation.

C. Forecasting Integration

Listing 3 retrieves Colombia's national unemployment rate (308 monthly observations, January 2001 to August 2026) and fits four forecasting models from the tidyverts ecosystem [6]–[8], including `NNETAR()`, a feed-forward neural network autoregression model, on top of the tidy output of `banrep_data()`, requiring only a single conversion step (`as_tsibble()`) to bridge the two ecosystems.

Listing 3. Downloading and forecasting an indicator

```
library(banrepstats); library(fable)
d <- banrep_data(
  "mercado_laboral_poblacion_tasa_ocupacion_
      desempleo"
) |>
  dplyr::filter(id == "Tasa␣de␣desempleo␣-␣total␣
      nacional") |>
  dplyr::mutate(mes = tsibble::yearmonth(date)) |>
  tsibble::as_tsibble(index = mes)

fit <- d |> model(
  arima = ARIMA(value),
  ets = ETS(value),
  nnetar = NNETAR(value),
  prophet = fable.prophet::prophet(value ~
    season(period = 12, order = 5,
         type = "multiplicative"))
)
fit |> forecast(h = "18␣months") |> autoplot(d)
```

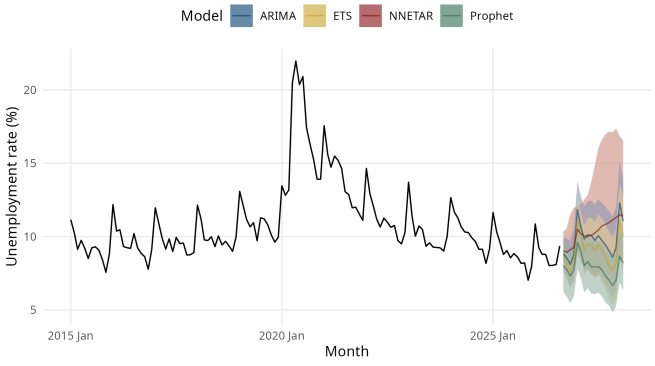


Fig. 2. 18-month forecast of Colombia's national unemployment rate, produced from data retrieved with `banrep_data()`. Models fitted on the full sample; shaded areas are 80% prediction intervals; history shown from 2015.

Fig. 2 shows the resulting 18-month forecast. Producing this plot requires no manual data wrangling beyond the single `tsibble` conversion, illustrating the package's goal: making Banrep data a first-class citizen of standard R analysis pipelines.

D. A Second Case: Quarterly GDP Growth

To confirm that the same workflow generalizes across frequencies, we repeat the exercise on Colombia's quarterly year-on-year real GDP growth rate (2006Q1–2026Q2, 82 observations), whose code is found with `search_banrep("PIB")`. Only the conversion step and the seasonal settings change: the index is built with `yearquarter()` instead of `yearmonth()`, and Prophet uses an additive seasonality of period four and order two instead of a multiplicative one of period twelve and order five. In both examples the seasonal period is matched to the frequency of the data. Fig. 3 shows the resulting two-year forecast.

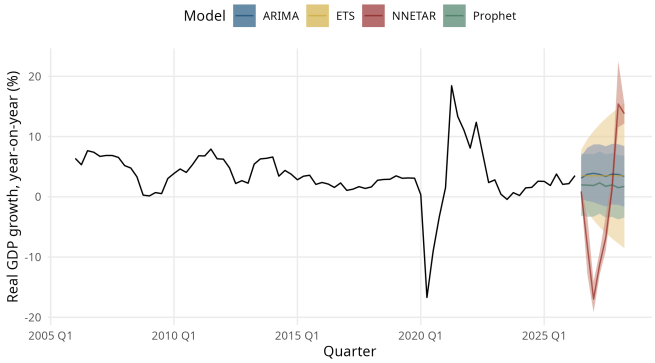


Fig. 3. Two-year forecast of Colombia's quarterly year-on-year GDP growth. Models fitted on the full sample; shaded areas are 80% prediction intervals.

VIII. Forecast Accuracy Evaluation

The purpose of this section is to show that the output of `banrep_data()` supports a complete forecasting workflow, including out-of-sample evaluation, rather than to establish which model is best: rankings depend on the indicator, the sample and the evaluation window, and a single holdout per series is not a basis for general conclusions about the models.

We hold out the last 10 quarters of GDP growth (2024Q1–2026Q2; 72 training observations, 2006Q1–2023Q4) and the last 10 months of the unemployment rate (November 2025 to August 2026; 298 training observations, January 2001 to October 2025), refit each model on the training data, and score the forecasts against the held-out observations with `fabletools::accuracy()`. Table IV reports the root-mean-squared error (RMSE), the mean absolute error (MAE), the mean absolute percentage error (MAPE) and the mean absolute scaled error (MASE) [7].

TABLE IV
Out-of-sample forecast accuracy by model and indicator

Indicator	Model	RMSE	MAE	MAPE	MASE
Unemployment	ARIMA	0.56	0.49	5.7	0.39
Unemployment	ETS	1.06	0.96	11.5	0.77
Unemployment	NNETAR	1.23	1.12	13.6	0.89
Unemployment	Prophet	0.81	0.73	8.9	0.58
GDP growth	ARIMA	2.07	1.78	149.2	0.43
GDP growth	ETS	1.79	1.60	83.0	0.39
GDP growth	NNETAR	4.60	4.26	334.7	1.04
GDP growth	Prophet	1.02	0.80	115.3	0.20

RMSE and MAE in percentage points; MAPE in %.

For the unemployment rate, ARIMA attains the lowest error on all four metrics, consistent with a smooth, strongly autocorrelated monthly series; Prophet ranks second, followed

by ETS, with `NNETAR()` last. For GDP growth the ranking changes: Prophet attains the lowest RMSE, MAE and MASE, followed by ETS and ARIMA, and `NNETAR()` performs worst of all four models. MAPE is unreliable for this indicator, because realized growth in the test window comes as close to zero as 0.22 percentage points, inflating every model's percentage error; this is why we also report MASE, which scales errors by the in-sample error of a seasonal naive forecast and stays well defined near zero, and ranks the GDP models in the same order as RMSE and MAE.

A plausible explanation for the poor GDP result of `NNETAR()` is overfitting. The automatically selected specification, NNAR$(9,1,5)_4$, has nine lagged inputs and five hidden units, i.e. 56 weights per network, estimated from only 63 training patterns, and its training sample contains the 2020 pandemic collapse and rebound. Two observations are consistent with this explanation: its in-sample RMSE (0.37) is twelve times smaller than its out-of-sample RMSE (4.60), whereas the other three models have out-of-sample errors below their in-sample errors; and its forecast in Fig. 3 reproduces a pandemic-like collapse and rebound that no other model predicts. For unemployment, where `NNETAR()` selects a much smaller NNAR$(1,1,2)_{12}$ network with nearly 300 training observations, the gap between in-sample and out-of-sample error is small. These results do not prove overfitting, but they make it the most likely explanation.

Reproducibility. The data were retrieved on October 24, 2026, and are frozen in the replication materials, since Banrep revises and extends its series. The results were produced with R 4.5.2, banrepstats 0.1.1 (whose code is unchanged in version 1.1.0), `fable` 0.5.0, `fabletools` 0.8.0, `fable.prophet` 0.1.0 and `tsibble` 1.2.0, with the random seed 20261024 reset before each set of models. The script `paper/forecast_eval.R` regenerates Table IV and Figs. 1–3.

IX. Discussion and Limitations

banrepstats depends on an internal API that Banrep does not commit to maintaining, so future changes to the SUAMECA application could break the package without notice. We mitigate this risk at four levels. First, the package detects changes instead of hiding them: responses are checked against the fields it relies on, and empty or malformed responses stop with an explicit error. Second, the offline test suite pins down the expected format of each response, while the live tests check that the real service still matches it. Third, the weekly monitoring job of the package repository runs those live tests, so a breaking change is noticed within days and fixed in a new release; the blank-slug change described in Section IV-C is an example of the kind of change this is meant to catch. Fourth, the bundled `banrep_cache` dataset gives callers an offline fallback for catalog lookups. None of these measures can prevent a breaking change, and a documented, versioned public API from Banrep would remain the most robust solution. The endpoint also returns each indicator's full history with no server-side date filtering; the `start` and `end` arguments of `banrep_data()` filter after the download, which is convenient but does not reduce the amount of data transferred. Field and category names throughout the catalog are in Spanish, which is faithful to the source but may require translation layers for non-Spanish-speaking users.

X. Conclusion

We presented banrepstats, an R package that turns Banco de la República's internal statistics API into a small, tidy, and forecasting-ready interface for R users. Beyond the package itself, we documented four concrete failure modes of consuming an undocumented government API – silent header-based filtering, timezone-dependent timestamp drift, unstable or non-unique identifiers, and an incomplete TLS certificate chain – that we expect recur when wrapping similar open-data sources elsewhere in the region. We showed that the package retrieves the full catalog and reproduces the portal's official exports exactly. Future work includes CRAN distribution, English-language aliases for catalog fields, and working with Banrep toward a documented public API.

Acknowledgment

The authors thank the maintainers of the open-source R packages this project builds on, in particular `data.table`, `httr2`, and the tidyverts forecasting ecosystem.

References

[1] E. Blondel, "rsdmx: Tools for Reading SDMX Data and Metadata," R package version 0.6-5, Comprehensive R Archive Network (CRAN), 2025.

[2] W. Freitas, "rbcb: R Interface to Brazilian Central Bank Web Services," R package version 0.1.14, Comprehensive R Archive Network (CRAN), 2024.

[3] M. Vargas Sepulveda, "wbstats: Programmatic Access to the World Bank API," Oak Ridge National Laboratory, Oak Ridge, TN, 2025. [Online]. Available: https://doi.org/10.11578/dc.20171025.1827

[4] L. Lahti, J. Huovari, M. Kainu, and P. Biecek, "Retrieval and analysis of Eurostat open data with the eurostat package," *The R Journal*, vol. 9, no. 1, pp. 385–392, 2017.

[5] T. Barrett, M. Dowle, A. Srinivasan, J. Gorecki, M. Chirico, T. Hocking, B. Schwendinger, and I. Krylov, "data.table: Extension of 'data.frame'," R package version 1.18.6.1, Comprehensive R Archive Network (CRAN), 2026.

[6] M. O'Hara-Wild, R. Hyndman, and E. Wang, "fable: Forecasting Models for Tidy Time Series," R package version 0.5.0, Comprehensive R Archive Network (CRAN), 2026.

[7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne: OTexts, 2021.

[8] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[9] H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*, 2nd ed. New York: Springer-Verlag, 2016.

[10] H. Wickham, "httr2: Perform HTTP Requests and Process the Responses," R package version 1.3.0, Comprehensive R Archive Network (CRAN), 2026.

[11] H. Wickham, "testthat: Get Started with Testing," *The R Journal*, vol. 3, no. 1, pp. 5–10, 2011.

[12] J. Hester, "covr: Test Coverage for Packages," R package version 3.6.5, Comprehensive R Archive Network (CRAN), 2025.